CS-UY 2214 — Verilog tutorial

1 Introduction

We're going to develop combinatorial circuits using the Verilog hardware description language. Circuits are broken into *modules*, where each module has well-defined *ports* that provide input and output signal to and from the module's implementation. Modules may contain other modules, and may be connected to other modules (analogously to how, in software engineering, a class may contain instances of other classes and may invoke methods on other classes).

In addition to writing circuits that represent modules, we'll write a *test bench* module, that will simply demonstrate another module by providing it with various inputs. We can run our test bench in a *simulator* that will let us examine the behavior of circuit almost as if it were implemented onto actual silicon. And finally, we can view the results of the simulation by graphing the values of the ports of the module.

2 Instructions

These instructions walk you through the testing of a small circuit in Verilog. We assume that you have access to a Linux machine or virtual machine running Debian or Ubuntu.

2.1 Setup

If you are using the virtual machine on Anubis or Vital, all tools should already be installed for you. For Anubis, please be sure to use the playground "CompArch / PL with Webtop" in order to be able to run GTKWave.

Otherwise, from your Linux machine's command prompt, install Icarus Verilog and GTKWave with the following `bash` command:

```
sudo apt-get install iverilog gtkwave
```

2.2 Our first circuit

Let's write some code. You can enter the following code yourself (using a text editor such as gedit, Sublime Text, vim, Emacs, pico, etc), or download the code in the attached file `verilog_tutorial.zip`.

Create the following file and name it `test.v`. This file describes the circuit that we are developing.

```
module test(A, B, C, D, E);

   output D, E;
   input  A, B, C;
   wire   w1;

   and G1(w1, A, B);
   not G2(E, C);
   or  G3(D, w1, E);

endmodule
```

In the above code, we use module instance syntax to create instances of an AND, NOT, and OR gate, and to connect their ports. Equivalently, we could define our module using this traditional bitwise operator syntax:

```
module test(A, B, C, D, E);

   output D, E;
   input  A, B, C;
   wire   w1;

   assign w1 = A & B;
   assign E = ~C;
   assign D = w1 | E;

endmodule
```

or, more concisely, like this:

```
module test(A, B, C, D, E);

   output D, E;
   input  A, B, C;

   assign E = ~C;
   assign D = (A & B) | E;

endmodule
```

We could also use the `?:` conditional operator to express a multiplexer. This operator has the syntax `condition ? value1 : value2` and the result is equal to `value1` if `condition` is true, otherwise the result is equal to `value2`. Therefore, instead of using a `not` gate to implement the value of output `E`, we could use a multiplexer, like this:

```
module test(A, B, C, D, E);

   output D, E;
   input  A, B, C;

   assign E = C ? 0 : 1;     // if C is 1, E is 0; if C is 0, E is 1
   assign D = (A & B) | E;

endmodule
```

Regardless of the implementation details, in every combinatorial Verilog module, each output port must be on the left-hand side of exactly one `assign` statement.

2.3 Testing the circuit with a test bench

Create the following file and name it `test_tb.v`. This file describes a test bench for evaluating the circuit in the previous file. It just runs through all the various possible inputs.

```
`timescale 1ns/100ps
`include "test.v"

module test_tb;

   wire A, B, C, D, E;
   integer k=0;

   assign {A,B,C} = k;
   test the_circuit(A, B, C, D, E);

   initial begin

      $dumpfile("test.vcd");
      $dumpvars(0, the_circuit);

      for (k=0; k<8; k=k+1)
        #10 $display("A=",A," B=",B," C=",C," D=",D," E=",E);

      $finish;

   end

endmodule
```

To compile the Verilog code, run this `bash` command:

```
iverilog -o test.vvp test_tb.v
```

Now run the compiled code in the simulator with this `bash` command:

```
vvp test.vvp
```

You should get output like this:

```
VCD info: dumpfile test.vcd opened for output.
A=0 B=0 C=0 D=1 E=1
A=0 B=0 C=1 D=0 E=0
A=0 B=1 C=0 D=1 E=1
A=0 B=1 C=1 D=0 E=0
A=1 B=0 C=0 D=1 E=1
A=1 B=0 C=1 D=0 E=0
A=1 B=1 C=0 D=1 E=1
A=1 B=1 C=1 D=1 E=0
```

Note that the output given shows us the state of various wires in our circuit at different times. It shows us that in the first clock cycle, A, B, C are zero, while D and E are one. Then, when we change C to one, both D and E become zero. By examining the output of the test bench, we can determine if our circuit is producing the desired result.

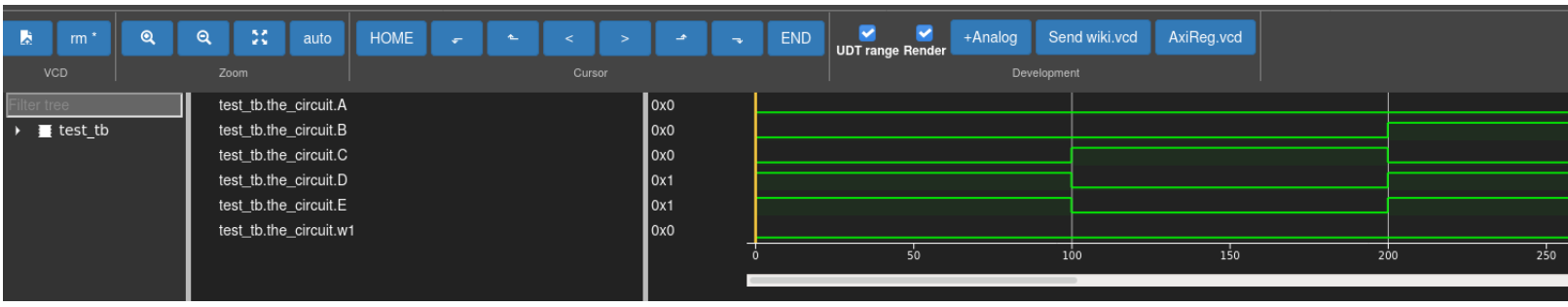
As noted in the first line of output, besides printing out the result of various wires, the test bench will also generate the file `test.vcd`, which captures the states of the system. We will use this file in the next step.

2.4 Waveform analysis

We've run the code, but to get a better view of what happened, let's view the circuit simulation. For this we can use either GTKWave (installed in Linux) or web-based wave viewer such as fliplot.

Analysis with fliplot If you're using fliplot, follow these instructions:

- Open `https://fliplot.nickelp.ro/` in your web browser.
- Select Open from fliplot's File menu and load your `test.vcd` file.
- You should now see this:



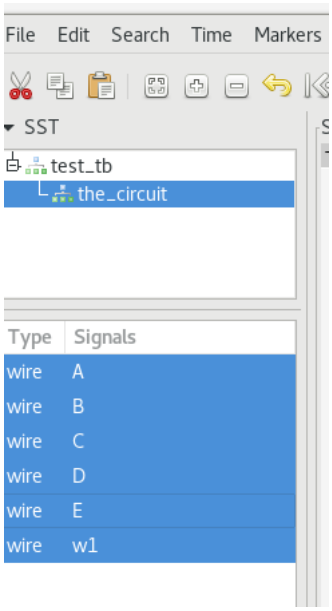
For this simple circuit, this result is fine. However, for more complicated circuits, such as those you'll work on for homework, we may need to selectively remove some wires from the display in order to make the output more concise.

Analysis with GTKWave If you're using GTKWave, follow these instructions:

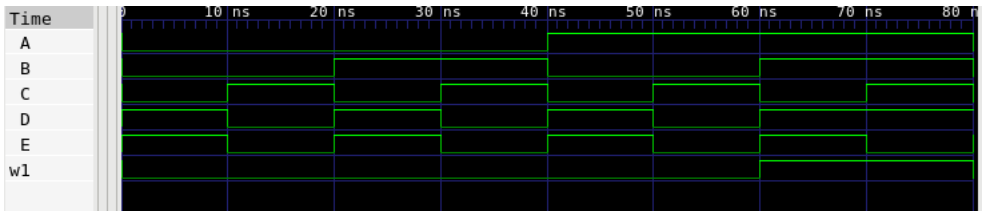
- First, run this `bash` command:

  ```
  gtkwave test.vcd
  ```

- Now, within the GTKWave user interface, click on the `test_tb` module, then on the circuit named `the_circuit` contained therein. The circuit's ports should appear in the panel below. Highlight all of the ports (`A` through `E` and `w1`) and click the Append button. Now the circuits should be added to the Signals panel on the right.



- Finally, press Alt-Shift-F or select Zoom Best Fit from the Time menu to show the complete wave.



Regardless of whether you're using fliplot or GTKWave, let's talk about this display. The diagram shows the state of six binary variables over time. Each green line corresponds to one of the variables; as the line moves between a high position and a low position, it indicates that the value changes between 0 and 1. Time advances from left to right. In the display above, we can see that the test bench varies the values of the module's input ports (A, B, and C), and the output ports change according to the rules we've defined. For example, you can see that E is always the opposite of C.

If a wire is displayed in red, or with the special value x, it means that that wire has an unknown value. This usually happens if you forget to assign a value to an output, or assign two conflicting values to the same output.

2.5 Synthesis

So far we've used Verilog to simulate a processor design. But what if we wanted to *synthesize* our design, that is, turn it into a chip?

First, if it's not already installed, let's install a synthesizer with this command:

```
sudo add-apt-repository universe
sudo apt-get install yosys
```

Now create a script file named `synth.ys` with the following content. Notice that the first line of the script refers to the `test.v` file that we created above.

```
read_verilog test.v
hierarchy; proc; opt; techmap; opt
write_verilog synth.v
show -format ps -prefix synth
```

Now run `yosys` with the following `bash` command:

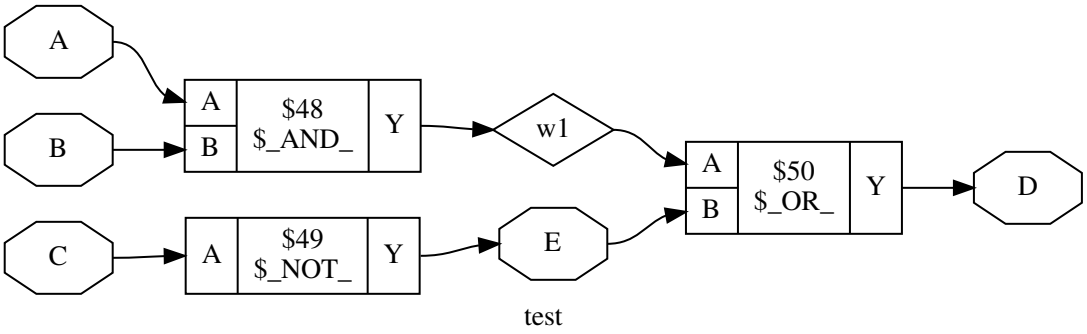
```
yosys synth.ys
```

If this command runs correctly, it will output two files of interest: `synth.v`, a gate-level Verilog file that can be used to generate hardware; and `synth.ps`, a visual representation of the gates. To view, use the following `bash` command:

```
xdg-open synth.ps
```

If you're using Anbuis, the above command won't work. You'll need to download `synth.ps` to your own computer and display it there.

You should see something like the following:



In the above diagram, note that the letters correspond to the ports and wires on the module declared in the `test.v` file; and that the logic gates are expressed as sub-modules (here labeled `$_NOT_`, `$_AND_`, and `$_OR_`.

This diagram is evidence that Verilog code can be expressed as logic gates. In this class, keep in mind that Verilog code does not express a program, but rather a hardware component, composed of physical gates.